

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1708

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 35%

QUESTIONS: 5

Duration : 1 Hour
Total Marks:50

Annexure A

Analytical Problem Solving

Code of Conduct:

*I **Jannathul Firdaus. N (192573025)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

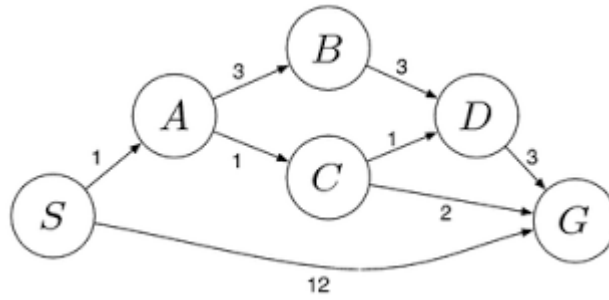
Signature of the student: Jannathul Firdaus. N

Annexure A

Analytical Problem Solving

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.
2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.
 - i. What are the percept's for this agent?
 - ii. Characterize the operating environment.
 - iii. What are the actions the agent can take?
 - iv. How can one evaluate the performance of the agent?
 - v. What sort of agent architecture do you think is most suitable for this agent? Why?
3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.
4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.
5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse S to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.

The delivery network is represented as a weighted graph:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem

1. Water Jug Problem – 4-Gallon and 3-Gallon Jugs

We are given a 4-gallon jug and a 3-gallon jug with no measurement markings. A pump can fill either jug, water can be discarded, and water can be transferred between the two jugs. The objective is to obtain exactly 2 gallons in the 4-gallon jug.

Problem Formulation

- State representation: (x, y) , where x is the amount in the 4-gallon jug and y is the amount in the 3-gallon jug.
- Initial state: $(0, 0)$.
- Goal state: any state in which the 4-gallon jug contains exactly 2 gallons, i.e., $(2, y)$.
- Actions: fill a jug, empty a jug, or pour water from one jug into the other until the source is empty or the destination is full.

Solution Steps

Step	Operation	4-Gallon Jug	3-Gallon Jug
1	Start	0	0
2	Fill the 3-gallon jug	0	3
3	Pour 3-gallon jug into 4-gallon jug	3	0
4	Fill the 3-gallon jug again	3	3
5	Pour into 4-gallon jug until it is full	4	2
6	Empty the 4-gallon jug	0	2
7	Pour the remaining 2 gallons into the 4-gallon jug	2	0

Final state: $(2, 0)$. Therefore, exactly 2 gallons are obtained in the 4-gallon jug.

Interpretation

This is a state-space search problem. Each pair (x, y) represents a state, while filling, emptying and transferring water are state-transition actions. The solution systematically reaches the required goal without using any measuring device.

2. Mars Rover – Intelligent Agent Analysis

A Mars rover is an intelligent robotic agent that senses the Martian environment, plans and performs navigation and scientific activities, collects or analyzes samples, and communicates scientific and engineering information to Earth.

2.1 Percepts for the Agent

Sensor / Source	Typical Percept
Cameras	Images of terrain, rocks, obstacles and surroundings
Spectrometers / scientific instruments	Chemical and mineral composition
Laser instruments	Properties of rocks and selected targets
Inertial and wheel sensors	Orientation, movement and wheel rotation
Navigation sensors	Terrain and obstacle information
Environmental sensors	Temperature, pressure, wind and dust conditions
Sample-analysis instruments	Physical and chemical characteristics of samples
Communication system	Commands and information exchanged with Earth

2.2 Operating Environment

Property	Characterization	Reason
Observability	Partially observable	The rover cannot observe the entire Martian environment at once.
Agents	Primarily single-agent	The rover operates autonomously, although Earth-based operators provide commands.
Determinism	Uncertain / non-deterministic	Wheel slip, terrain variation, dust and unexpected conditions introduce uncertainty.
Sequentiality	Sequential	Current actions affect future states and mission options.
Dynamics	Dynamic	Environmental and rover conditions can change over time.
Continuity	Largely continuous	Position, orientation, speed and environmental measurements vary continuously.
Knowledge	Partially unknown	The complete terrain and geological properties are not known beforehand.

2.3 Actions the Agent Can Take

- Move forward or backward and turn left or right.
- Stop, avoid obstacles and select safe routes.
- Capture images and perform scientific measurements.
- Approach scientific targets and collect or drill samples.

- Analyze and store samples.
- Transmit scientific and engineering data to Earth.
- Manage energy and enter a safe state when faults or hazardous conditions are detected.

2.4 Performance Evaluation

The rover can be evaluated using a performance measure involving:

- Scientific value: quality and quantity of useful observations and samples.
- Mission progress: completion of assigned exploration objectives.
- Navigation: successful movement toward target locations and obstacle avoidance.
- Resource efficiency: energy, time and communication-bandwidth usage.
- Safety and reliability: avoidance of collisions, equipment protection and fault recovery.

2.5 Suitable Agent Architecture

Recommended architecture: a model-based, utility-based agent with learning capabilities.

A model-based component is necessary because the rover cannot directly observe the complete environment. It maintains an internal model of terrain, obstacles, location, scientific targets and rover status. A utility-based component helps balance scientific value, safety, energy and travel time. Learning can improve decisions from previous observations and experience. Therefore, this architecture is appropriate for the rover's uncertain, partially observable and communication-delayed environment.

3. 8-Queens Problem – Search Space and Problem Formulation

The 8-Queens problem requires placing eight queens on an 8×8 chessboard so that no two queens attack one another. A queen attacks along its row, column and diagonals.

Problem Components

Component	Formulation
Initial state	Empty board: []
State representation	$[x_1, x_2, \dots, x_8]$, where x_i is the row of the queen in column i .
Actions	Place a queen in a safe square in the next column.
Transition model	Add one queen to the next column without creating a row or diagonal conflict.
Goal test	Eight queens have been placed and no pair attacks each other.
Path cost	If each placement costs 1, every complete solution has cost 8; the main objective is to find a valid solution.

Constraint Conditions

For queens at positions (r_i, c_i) and (r_j, c_j) , a valid configuration must satisfy:

- $r_i \neq r_j$ (different rows)
- $c_i \neq c_j$ (different columns)
- $|r_i - r_j| \neq |c_i - c_j|$ (different diagonals)

Example valid solution: [1, 5, 8, 6, 3, 7, 2, 4]

This representation places one queen in each column and satisfies the row and diagonal constraints.

Search Space

If one queen is assigned to each column, there are $8^8 = 16,777,216$ possible row assignments before constraints are applied. If the one-queen-per-row condition is also imposed, there are $8! = 40,320$ permutations. Backtracking reduces the effective search by rejecting partial configurations as soon as a conflict occurs.

Suitable Search Strategy – DFS with Backtracking

Depth-First Search with backtracking is suitable because it constructs a solution incrementally and immediately abandons a branch when the partial configuration cannot lead to a valid solution.

Pseudocode

```
NQueens(board, column):  
    if column > 8:  
        return SUCCESS  
    for each row from 1 to 8:  
        if Safe(board, row, column):  
            place queen at (row, column)  
            if NQueens(board, column + 1) == SUCCESS:  
                return SUCCESS  
            remove queen from (row, column)  
    return FAILURE
```

4. OLA Cab Booking – Problem-Solving Agent

A customer wants to travel from a source location to a destination using a cab-booking application. Several cab categories may be available, such as Micro, Mini, Sedan, Shared and Prime. Each option may differ in fare, travel time, comfort and availability.

Type of Problem-Solving Agent

Suitable agent: a utility-based goal-oriented agent.

The goal is to transport the customer from the source to the destination. Because several cab choices can satisfy the goal but differ in cost, estimated time, comfort and availability, a utility-based agent can compare alternatives according to the customer's preferences.

Problem Formulation

Component	Description
Initial state	Customer is at the source location.
Goal state	Customer reaches the destination.
Actions	Enter locations, obtain cab options, compare them, select a cab, book it, track the driver and complete the trip.
State information	Source, destination, cab type, fare, estimated time and availability.
Utility factors	Cost, travel time, comfort, availability and customer preference.

Pseudocode

```
OLA_CAB_BOOKING():
  INPUT source
  INPUT destination
  IF source == destination:
    DISPLAY 'Source and destination cannot be the same'
    RETURN
  availableCabs ← GetAvailableCabs(source, destination)
  IF availableCabs is empty:
    DISPLAY 'No cabs available'
    RETURN
  FOR each cab in availableCabs:
    fare ← EstimateFare(cab, source, destination)
    time ← EstimateTravelTime(source, destination)
    availability ← CheckAvailability(cab)
    utility[cab] ← CalculateUtility(fare, time, cab.comfort, availability)
  selectedCab ← SelectCabWithHighestUtility(utility)
  DISPLAY selectedCab, fare and estimated time
  IF CustomerConfirmsBooking():
    booking ← BookCab(selectedCab, source, destination)
    IF booking is successful:
      DISPLAY 'Cab booked successfully'
      TrackCab(booking)
    ELSE:
      DISPLAY 'Booking failed'
  ELSE:
    DISPLAY 'Booking cancelled'
```

Interpretation

The agent should not treat the task as merely selecting the cheapest cab. The complete problem is to choose an available service that satisfies the travel goal while accounting for the customer's selected preferences and relevant service attributes.

5. Logistics Network – Uniform Cost Search (UCS)

The logistics network is a weighted graph. Warehouse S is the starting node and warehouse G is the goal. Each directed edge has an associated travel cost. The objective is to find the path from S to G with the minimum total cost.

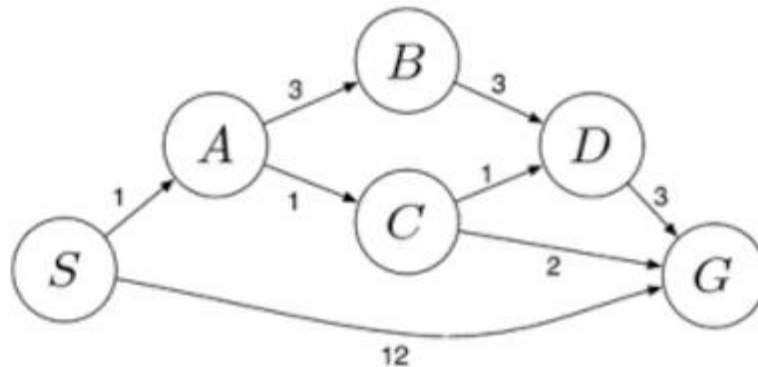


Figure: Given weighted delivery network

Problem Formulation

Component	Description
Initial state	S
Goal state	G
Actions	Travel along an available directed edge to a connected warehouse.
Transition cost	The numerical weight shown on the selected edge.
Path cost	Sum of all edge costs along the route from S to G.
Objective	Find the least-cost path from S to G.

Why Uniform Cost Search?

UCS is appropriate because the objective is minimum cumulative cost. At every iteration, UCS expands the frontier node with the smallest path cost $g(n)$, where $g(n)$ is the total cost from S to that node. For non-negative edge costs, UCS is optimal: when G is removed from the priority queue for expansion, the corresponding path is a least-cost path.

UCS Calculation

Step	Expanded Node	Cumulative Cost $g(n)$	Priority Queue / Frontier after expansion
1	S	0	A(1), G(12)
2	A	1	C(2), B(4), G(12)
3	C	2	D(3), B(4), G(4), D(3) → same node

			at lower cost replaces any higher- cost route
4	D	3	G(4), B(4), G(6)
5	G	4	Goal reached

For clarity, after expanding C, the route $S \rightarrow A \rightarrow C$ gives D a cumulative cost of 3 and G a cumulative cost of 4. The alternative route $S \rightarrow A \rightarrow B \rightarrow D$ would give D a cost of 7, so it is discarded because a cheaper route to D is already known. The direct route $S \rightarrow G$ has cost 12.

Detailed Cost Calculation

Path / Route	Calculation	Total Cost
$S \rightarrow A$	1	1
$S \rightarrow A \rightarrow C$	$1 + 1$	2
$S \rightarrow A \rightarrow C \rightarrow D$	$1 + 1 + 1$	3
$S \rightarrow A \rightarrow C \rightarrow G$	$1 + 1 + 2$	4
$S \rightarrow A \rightarrow B$	$1 + 3$	4
$S \rightarrow A \rightarrow B \rightarrow D$	$1 + 3 + 3$	7
$S \rightarrow G$	12	12
$S \rightarrow A \rightarrow C \rightarrow D \rightarrow G$	$1 + 1 + 1 + 3$	6

Optimal path: $S \rightarrow A \rightarrow C \rightarrow G$

Minimum total cost: $1 + 1 + 2 = 4$

Interpretation of Result

Uniform Cost Search first considers the cumulative cost rather than the number of edges. The direct route $S \rightarrow G$ costs 12, while $S \rightarrow A \rightarrow C \rightarrow G$ costs only 4. The route through D costs 6, and the route through B and D costs 7. Therefore, the least-cost delivery route is $S \rightarrow A \rightarrow C \rightarrow G$ with a total travel cost of 4 cost units.